

Enkapsulasi Pada Pemrograman Berorientasi Objek

1. Kompetensi

Setelah melakukan percobaan pada modul ini, mahasiswa memahami konsep:

1. Konstruktor
2. Akses Modifier
3. Atribut/method pada class
4. Instansiasi atribut/method
5. Setter dan getter
6. Memahami notasi pada UML Class Diagram

2. Pendahuluan

Pada pertemuan pertama dan kedua telah dibahas konsep dasar dari pemrograman berbasis objek (PBO), perbedaan antara pemrograman berbasis objek dengan pemrograman struktural, dan telah dibahas konsep class dan object pada PBO. Selanjutnya pada modul ini akan dibahas konsep enkapsulasi pada PBO dan notasi yang ada pada UML Class diagram.

2.1 Enkapsulasi

Pada modul pertama telah dijabarkan definisi dari enkapsulasi sebagai berikut:

Enkapsulasi disebut juga dengan **information-hiding**. Dalam berinteraksi dengan objek, seringkali kita tidak perlu mengetahui kompleksitas yang ada didalamnya. Hal ini akan lebih mudah dipahami jika kita membayangkan atau menganalisa objek yang ada disekitar kita, misalnya objek sepeda, ketika kita mengganti gear pada sepeda, kita tinggal menekan tuas gear yang ada di grip setang sepeda saja. Kita tidak perlu mengetahui bagaimana cara gear berpindah secara teknis. Contoh objek lain misalnya mesin penghisap debu (vacuum cleaner), ketika kita mencolokkan kabel vacum cleaner dan menyalakan saklarnya maka mesin tersebut siap digunakan untuk menghisap debu. Dalam proses tersebut kita tidak mengetahui proses rumit yang terjadi ketika mengubah listrik menjadi tenaga dari vacuum cleaner. Dalam contoh diatas vacum cleaner dan sepeda telah menerapkan enkapsulasi atau disebut juga **information-hiding atau data hiding** karena menyembunyikan detail proses suatu objek dari pengguna.

2.2 Konstruktor

Konstruktor mirip dengan method cara deklarasinya akan tetapi tidak memiliki tipe return. Dan konstruktor dieksekusi ketika instan dari objek dibuat. Jadi setiap kali sebuah objek dibuat dengan keyword `new()` maka konstruktor akan dieksekusi. Cara untuk membuat konstruktor adalah sebagai berikut:

1. Nama konstruktor harus sama dengan nama class
2. Konstruktor tidak memiliki tipe data return
3. Konstruktor tidak boleh menggunakan modifier `abstract`, `static`, `final`, dan `synchronized`

Note: Di java kita dapat memiliki konstruktor dengan modifier `private`, `protected`, `public` or default.

Secara umum, konstruktor dideklarasikan dengan modifier `public` untuk menjamin bahwa kelas tersebut dapat diinstansiasi secara bebas oleh kelas driver atau komponen eksternal lain di dalam sistem. Namun, dalam arsitektur pemrograman berorientasi objek tingkat lanjut, penggunaan modifier lain diizinkan untuk memenuhi kebutuhan desain spesifik:

1. **Private Constructor:** Digunakan untuk menutup total akses instansiasi dari luar kelas. Pola ini umumnya diterapkan pada implementasi **Singleton Pattern** (memastikan hanya ada satu objek yang hidup di memori) atau pada **Utility Class** yang hanya berisi sekumpulan method statis (seperti kelas `Math` bawaan Java), sehingga pembuatan objek kelas tersebut menjadi tidak relevan.
2. **Default/Protected Constructor:** Digunakan ketika instansiasi objek ingin dibatasi hanya untuk kelas-kelas dalam satu package yang sama, atau hanya boleh diinisialisasi oleh kelas turunannya (**subclass**) dalam koridor pewarisan (**inheritance**).

2.3 Akses Modifier

Terdapat 2 tipe modifier di java yaitu : akses modifier dan non-access modifier. Dalam hal ini kita akan fokus pada akses modifier yang berguna untuk mengatur akses method, class, dan constructor.

Terdapat 4 akses modifier yaitu:

1. `private` – hanya dapat diakses di dalam kelas yang sama
2. `default` – hanya dapat diakses di dalam package yang sama
3. `protected` – dapat diakses di luar package menggunakan subclass (membuat inheritance)
4. `public` – dapat diakses dari mana saja

Detail akses modifier dapat dilihat pada Tabel 1.1.

Tabel 1. 1 Detail Access Modifier

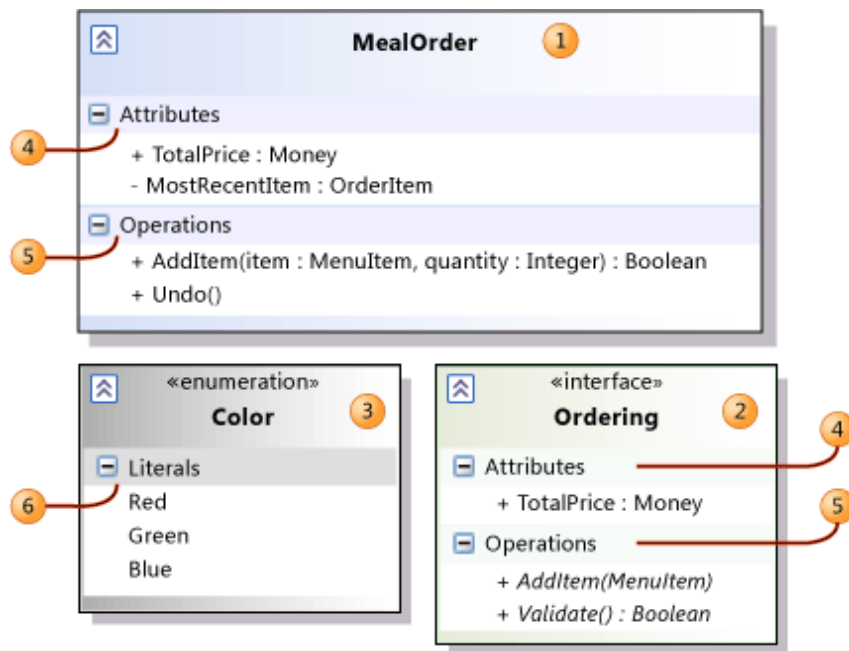
Access Modifier	Notation	within class	within package	outside package by subclass only	outside package
Private	- (minus)	Y	N	N	N
Default	~ (tilde)	Y	Y	N	N
Protected	# (pagar)	Y	Y	Y	N
Public	+ (plus)	Y	Y	Y	Y

2.4 Getter dan Setter

Getter adalah public method dan memiliki tipe data return, yang berfungsi untuk mendapatkan nilai dari atribut private. Sedangkan setter adalah public method yang tidak memiliki tipe data return, yang berfungsi untuk memanipulasi nilai dari atribut private.

2.5 Notasi UML Class Diagram

Secara umum bentuk UML class diagram adalah seperti pada Gambar 1.1



Gambar 1. 1 UML Class Diagram

Keterangan :

1. Class
2. Interface
3. Enumeration – adalah tipe data yang memiliki nilai atau literal yang terbatas.
4. Attributes
5. Method
6. Literals

Notasi akses modifier pada UML class diagram adalah sebagai berikut:

1. Tanda plus (+) untuk public
2. Tanda pagar (#) untuk protected
3. Tanda minus (-) untuk private
4. Tanda tilde (~) atau tanpa tanda untuk default

3. Percobaan

3.1 Percobaan 1 - Enkapsulasi

Didalam percobaan enkapsulasi, buatlah class **Motor** yang memiliki atribut kecepatan dan kontakOn, dan memiliki method `printStatus()` untuk menampilkan status motor. Seperti berikut

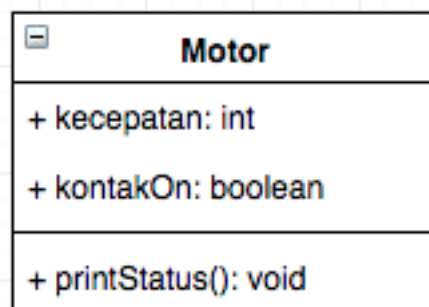
1. Buka Netbeans, buat project **MotorEncapsulation**.
2. Buat class **Motor**. Klik kanan pada package **motorencapsulation** – New – Java Class.
3. Ketikkan kode class **Motor** dibawah ini.

```

1  package motorencapsulation;
2
3  public class Motor {
4      public int kecepatan = 0;
5      public boolean kontakOn = false;
6
7      public void printStatus(){
8          if (kontakOn == true){
9              System.out.println("Kontak On");
10         }
11         else{
12             System.out.println("Kontak Off");
13         }
14         System.out.println("Kecepatan " + kecepatan+"\n");
15     }
16 }

```

bentuk UML class diagram class Motor adalah sebagai berikut:



4. Kemudian buat class MotorDemo, ketikkan kode berikut ini.

```

1  package motorencapsulation;
2
3  public class MotorDemo {
4      public static void main(String[] args) {
5          Motor motor = new Motor();
6          motor.printStatus();
7          motor.kecepatan = 50;
8          motor.printStatus();
9      }
10 }

```

5. Hasilnya adalah sebagai berikut:

```
run:
Kontak Off
Kecepatan 0

Kontak Off
Kecepatan 50

BUILD SUCCESSFUL (total time: 0 seconds)
```

Dari percobaan 1 - enkapsulasi, menurut anda, adakah yang janggal?

Yaitu, kecepatan motor tiba-tiba saja berubah dari 0 ke 50. Lebih janggal lagi, posisi kontak motor masih dalam kondisi OFF. Bagaimana mungkin sebuah motor bisa sekejap berkecepatan dari nol ke 50, dan itupun kunci kontaknya OFF?

Nah dalam hal ini, akses ke atribut motor ternyata tidak terkontrol. Padahal, objek di dunia nyata selalu memiliki batasan dan mekanisme bagaimana objek tersebut dapat digunakan. Lalu, bagaimana kita bisa memperbaiki class Motor diatas agar dapat digunakan dengan baik? Kita bisa pertimbangkan beberapa hal berikut ini:

1. Menyembunyikan atribut internal (kecepatan, kontakOn) dari pengguna (class lain)
2. Menyediakan method khusus untuk mengakses atribut.

Untuk itu mari kita lanjutkan percobaan berikut nya tentang Access Modifier.

3.2 Percobaan 2 - Access Modifier

Pada percobaan ini akan digunakan access modifier untuk memperbaiki cara kerja class Motor pada percobaan ke-1.

1. Ubah cara kerja class motor sesuai dengan UML class diagram berikut.



2. Berdasarkan UML class diagram tersebut maka class Motor terdapat perubahan, yaitu:
 - a. Ubah access modifier kecepatan dan kontakOn menjadi private

- b. Tambahkan method nyalakanMesin, matikanMesin, tambahKecepatan, kurangiKecepatan.

Implementasi class Motor adalah sebagai berikut:

```
1 package motorencapsulation;
2 public class Motor {
3     private int kecepatan = 0;
4     private boolean kontakOn = false;
5     public void nyalakanMesin(){
6         kontakOn = true;
7     }
8     public void matikanMesin(){
9         kontakOn = false;
10        kecepatan = 0;
11    }
12    public void tambahKecepatan(){
13        if (kontakOn == true){
14            kecepatan += 5;
15        }
16        else{
17            System.out.println("Kecepatan tidak bisa bertambah karena Mesin Off! \n");
18        }
19    }
20    public void kurangiKecepatan(){
21        if (kontakOn == true){
22            kecepatan -= 5;
23        }
24        else{
25            System.out.println("Kecepatan tidak bisa berkurang karena Mesin Off! \n");
26        }
27    }
28    public void printStatus(){
29        if (kontakOn == true){
30            System.out.println("Kontak On");
31        }
32        else{
33            System.out.println("Kontak Off");
34        }
35        System.out.println("Kecepatan " + kecepatan+"\n");
36    }
37 }
```

access modifier diubah private

method yang ditambahkan

3. Kemudian pada class MotorDemo, ubah code menjadi seperti berikut:

```

1 package motorencapsulation;
2
3 public class MotorDemo {
4     public static void main(String[] args) {
5         Motor motor = new Motor();
6         motor.printStatus();
7         motor.tambahKecepatan();
8
9         motor.nyalakanMesin();
10        motor.printStatus();
11
12        motor.tambahKecepatan();
13        motor.printStatus();
14
15        motor.tambahKecepatan();
16        motor.printStatus();
17
18        motor.tambahKecepatan();
19        motor.printStatus();
20
21        motor.matikanMesin();
22        motor.printStatus();
23    }
24 }

```

4. Hasilnya dari class MotorDemo adalah sebagai berikut:

```

run:
Kontak Off
Kecepatan 0
Kecepatan tidak bisa bertambah karena Mesin Off!
Kontak On
Kecepatan 0
Kontak On
Kecepatan 5
Kontak On
Kecepatan 10
Kontak On
Kecepatan 15
Kontak Off
Kecepatan 0
BUILD SUCCESSFUL (total time: 0 seconds)

```

Dari percobaan diatas, dapat kita amati sekarang atribut kecepatan tidak bisa diakses oleh pengguna dan diganti nilainya secara sembarangan. Bahkan ketika mencoba menambah kecepatan saat posisi kontak masih OFF, maka akan muncul notifikasi bahwa mesin OFF. Untuk mendapatkan kecepatan

yang diinginkan, maka harus dilakukan secara gradual, yaitu dengan memanggil method `tambahKecepatan()` beberapa kali. Hal ini mirip seperti saat kita mengendarai motor.

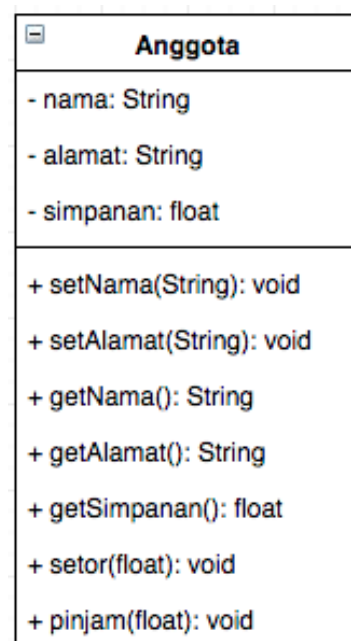
3.3 Pertanyaan

1. Pada class `TestMobil`, saat kita menambah kecepatan untuk pertama kalinya, mengapa muncul peringatan “Kecepatan tidak bisa bertambah karena Mesin Off!”?
2. Mengapa atribut kecepatan dan kontakOn diset private?
3. Ubah class `Motor` sehingga kecepatan maksimalnya adalah 100!

3.4 Percobaan 3 - Getter dan Setter

Misalkan di sebuah sistem informasi koperasi, terdapat class `Anggota`. Anggota memiliki atribut nama, alamat dan simpanan, dan method setter, getter dan setor dan pinjam. Semua atribut pada anggota tidak boleh diubah sembarangan, melainkan hanya dapat diubah melalui method setter, getter, setor dan tarik. Khusus untuk atribut simpanan tidak terdapat setter karena simpanan akan bertambah ketika melakukan transaksi setor dan akan berkurang ketika melakukan peminjaman/tarik.

1. Berikut ini UML class buatlah class `Mahasiswa` pada program:



2. Sama dengan percobaan 1 untuk membuat project baru
 - a. Buka Netbeans, buat project **KoperasiGetterSetter**.
 - b. Buat class **Anggota**. Klik kanan pada package **koperasigettersetter** – New – Java Class.
 - c. Ketikkan kode class `Anggota` dibawah ini.

```

1 package kopersigettersetter;
2 public class Anggota {
3     private String nama;
4     private String alamat;
5     private float simpanan;
6
7     public void setNama(String nama){
8         this.nama = nama;
9     }
10    public void setAlamat(String alamat){
11        this.alamat = alamat;
12    }
13    public String getNama(){
14        return nama;
15    }
16    public String getAlamat(){
17        return alamat;
18    }
19    public float getSimpanan(){
20        return simpanan;
21    }
22    public void setor(float uang){
23        simpanan +=uang;
24    }
25    public void pinjam(float uang){
26        simpanan -=uang;
27    }
28 }

```

setter, getter
nama dan alamat

getter
simpanan

Jika diperhatikan pada class Anggota, atribut nama dan alamat memiliki masing-masing 1 getter dan setter. Sedangkan atribut simpanan hanya memiliki getSimpanan() saja, karena seperti tujuan awal, atribut simpanan akan berubah nilainya jika melakukan transaksi setor() dan pinjam/tarik().

- Selanjutnya buatlah class KoperasiDemo untuk mencoba class Anggota.

```

1 package kopersigettersetter;
2 public class KoperasiDemo {
3     public static void main(String[] args) {
4         Anggota anggota1 = new Anggota();
5         anggota1.setNama("Iwan Setiawan");
6         anggota1.setAlamat("Jalan Sukarno Hatta no 10");
7         anggota1.setor(100000);
8         System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
9
10        anggota1.pinjam(5000);
11        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
12    }
13 }

```

- Hasil dari main method pada langkah ketiga adalah

```
run:
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Dapat dilihat pada hasil percobaan diatas, untuk mengubah simpanan tidak dilakukan secara langsung dengan mengubah atribut simpanan, melainkan melalui method setor() dan pinjam(). Untuk menampilkan nama pun harus melalui method getName(), dan untuk menampilkan simpanan melalui getSimpanan().

3.5 Percobaan 4 - Konstruktor, Instansiasi

- Langkah pertama percobaan 4 adalah ubah class KoperasiDemo seperti berikut

```
1 package koperasigettersetter;
2 public class KoperasiDemo {
3     public static void main(String[] args) {
4         Anggota anggota1 = new Anggota();
5         System.out.println("Simpanan " + anggota1.getName() + " : Rp " + anggota1.getSimpanan());
6
7         anggota1.setName("Iwan Setiawan");
8         anggota1.setAlamat("Jalan Sukarno Hatta no 10");
9         anggota1.setor(100000);
10        System.out.println("Simpanan " + anggota1.getName() + " : Rp " + anggota1.getSimpanan());
11
12        anggota1.pinjam(5000);
13        System.out.println("Simpanan " + anggota1.getName() + " : Rp " + anggota1.getSimpanan());
14    }
15 }
```

- Hasil dari program tersebut adalah sebagai berikut

```
run:
Simpanan null : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Dapat dilihat hasil running program, ketika dilakukan pemanggilan method getName() hasilnya hal ini terjadi karena atribut nama belum diset nilai defaultnya. Hal ini dapat ditangani dengan membuat konstruktor.

3. Ubah class Anggota menjadi seperti berikut

```
1 package kopersigettersetter;
2 public class Anggota {
3     private String nama;
4     private String alamat;
5     private float simpanan;
6
7     Anggota(String nama, String alamat){
8         this.nama = nama;
9         this.alamat = alamat;
10        this.simpanan = 0;
11    }
12
13    public void setNama(String nama){
14        this.nama = nama;
15    }
16    public void setAlamat(String alamat){
17        this.alamat = alamat;
18    }
19    public String getNama(){
20        return nama;
21    }
22    public String getAlamat(){
23        return alamat;
24    }
25    public float getSimpanan(){
26        return simpanan;
27    }
28    public void setor(float uang){
29        simpanan +=uang;
30    }
31    public void pinjam(float uang){
32        simpanan -=uang;
33    }
34 }
35
```

Pada class Anggota dibuat konstruktor dengan access modifier default yang memiliki 2 parameter nama dan alamat. Dan didalam konstruktor tersebut dipastikan nilai simpanan untuk pertama kali adalah Rp. 0.

4. Selanjutnya ubah class KoperasiDemo sebagai berikut

```
1 package kopersigettersetter;
2 public class KoperasiDemo {
3     public static void main(String[] args) {
4         Anggota anggota1 = new Anggota("Iwan", "Jalan Mawar");
5         System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
6
7         anggota1.setNama("Iwan Setiawan");
8         anggota1.setAlamat("Jalan Sukarno Hatta no 10");
9         anggota1.setor(100000);
10        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
11
12        anggota1.pinjam(5000);
13        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
14    }
15 }
```

Passing Parameter

5. Hasil dari program tersebut adalah sebagai berikut

```
run:
Simpanan Iwan : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Setelah menambah konstruktor pada class Anggota maka atribut nama dan alamat secara otomatis harus diset terlebih dahulu dengan melakukan passing parameter jika melakukan instansiasi class Anggota. Hal ini biasa dilakukan untuk atribut yang membutuhkan nilai yang spesifik. Jika tidak membutuhkan nilai spesifik dalam konstruktor tidak perlu parameter. Contohnya simpanan untuk anggota baru diset 0, maka simpanan tidak perlu untuk dijadikan parameter pada konstruktor.

3.6 Pertanyaan – Percobaan 3 dan 4

1. Apa yang dimaksud getter dan setter?
2. Apa kegunaan dari method getSimpanan()?
3. Method apa yang digunakan untuk menambah saldo?
4. Apa yang dimaksud konstruktor?
5. Sebutkan aturan dalam membuat konstruktor?
6. Apakah boleh konstruktor bertipe private?
7. Kapan menggunakan konstruktor dengan passing parameter?
8. Apa perbedaan inisialisasi atribut dan instansiasi atribut?
9. Apa perbedaan inisialisasi method dan instansiasi method?

4. Kesimpulan

Dari percobaan diatas, telah dipelajari konsep dari enkapsulasi, konstruktor, access modifier yang terdiri dari 4 jenis yaitu public, protected, default dan private. Konsep atribut dan method class yang ada di dalam blok code class dan konsep instansiasi atribut atau method. Cara penggunaan getter dan setter beserta fungsi dari getter dan setter. Dan juga telah dipelajari atau memahami notasi UML

5. Tugas

1. Cobalah program dibawah ini dan tuliskan hasil outputnya

```

public class EncapDemo
{
    private String name;
    private int age;

    public String getName()
    {
        return name;
    }

    public void setName(String newName)
    {
        name = newName;
    }

    public int getAge()
    {
        return age;
    }

    public void setAge(int newAge)
    {
        if(newAge > 30)
        {
            age = 30;
        }
        else
        {
            age = newAge;
        }
    }
}

public class EncapTest
{
    public static void main(String args[])
    {
        EncapDemo encap = new EncapDemo();
        encap.setName("James");
        encap.setAge(35);

        System.out.println("Name : " + encap.getName());
        System.out.println("Age : " + encap.getAge());
    }
}

```

2. Pada program diatas, pada class EncapTest kita mengeset age dengan nilai 35, namun pada saat ditampilkan ke layar nilainya 30, jelaskan mengapa.

Jawaban:

3. Ubah program diatas agar atribut age dapat diberi nilai maksimal 30 dan minimal 18.

Jawaban:

4. Pada sebuah sistem manajemen pergudangan kargo ekspedisi, terdapat class Kontainer yang memiliki atribut antara lain nomorResi, namaPemilik, kapasitasMaksimal (dalam kg), dan beratMuatanSaatIni. Kontainer dapat menerima tambahan muatan barang dengan batasan kapasitas maksimal yang telah ditentukan. Kontainer juga dapat diturunkan muatannya (bongkar muat). Ketika barang diturunkan, maka jumlah muatan saat ini akan berkurang sesuai dengan nominal berat yang dikeluarkan.

Buatlah class Kontainer tersebut, berikan atribut (private), method getter, dan konstruktor sesuai dengan kebutuhan arsitektur enkapsulasi. Uji dengan kelas driver TestLogistik berikut ini untuk memeriksa apakah manajemen state kelas Anda telah berjalan dengan benar:

```

1 public class TestLogistik {
2     public static void main(String[] args) {
3         Kontainer kontainerAlfa = new Kontainer("REQ-9988", "PT. Maju Bersama", 5000);
4
5         System.out.println("Nama Pemilik Kontainer: " + kontainerAlfa.getNamaPemilik());
6         System.out.println("Kapasitas Maksimal: " + kontainerAlfa.getKapasitasMaksimal() + " kg");
7
8         System.out.println(x: x: "\nMemasukkan muatan baru seberat 6.000 kg...");
9         kontainerAlfa.tambahMuatan(6000);
10        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
11
12        System.out.println(x: x: "\nMemasukkan muatan baru seberat 4.000 kg...");
13        kontainerAlfa.tambahMuatan(4000);
14        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
15
16        System.out.println(x: x: "\nMembongkar muat/menurunkan barang seberat 500 kg...");
17        kontainerAlfa.turunkanMuatan(500);
18        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
19
20        System.out.println(x: x: "\nMembongkar muat/menurunkan barang seberat 1.500 kg...");
21        kontainerAlfa.turunkanMuatan(1500);
22        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
23    }
24 }

```

Hasil yang diharapkan:

```

Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 6.000 kg...
Maaf, berat muatan melebihi kapasitas maksimal kontainer.
Berat muatan saat ini: 0.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Membongkar muat/menurunkan barang seberat 500 kg...
Berat muatan saat ini: 3500.0 kg

Membongkar muat/menurunkan barang seberat 1.500 kg...
Berat muatan saat ini: 2000.0 kg

```

Jawaban:

- Modifikasi soal kargo logistik di atas agar nominal berat muatan yang dibongkar/diturunkan dalam satu kali pemanggilan method turunkanMuatan() maksimal hanya boleh sebesar 50%

dari total berat muatan saat ini. Langkah ini diterapkan demi alasan keselamatan kerja operasional alat berat (crane). Jika operator mencoba menurunkan muatan melebihi batas 50% tersebut, sistem harus memblokir aksi dan memunculkan peringatan: "Maaf, demi keselamatan, pembongkaran muatan satu kali jalan tidak boleh melebihi 50% dari muatan saat ini!".

Jawaban:

6. Modifikasi kelas Main TestLogistik agar parameter jumlah berat barang yang dimasukkan (tambahMuatan) maupun berat barang yang dibongkar (turunkanMuatan) dapat menerima input nilai dinamis dari pengguna secara interaktif melalui terminal menggunakan utilitas `java.util.Scanner`.

Jawaban:

7. Sebuah aplikasi pemesanan tiket bioskop memerlukan kelas Tiket untuk mengelola data pemesanan secara aman. Kelas ini harus memiliki atribut private: `judulFilm` (String), `hargaDasar` (double), dan `statusPembayaran` (boolean).

Ketentuan pengesetan nilai objek:

- Konstruktor harus menerima parameter `judulFilm` dan `hargaDasar`. Nilai awal `statusPembayaran` selalu diset false (Belum Dibayar).
- Atribut `hargaDasar` tidak boleh bernilai negatif. Jika input yang dimasukkan kurang dari 0, otomatis set nilai default ke Rp 35.000.
- Sediakan method `lakukanPembayaran()` untuk mengubah `statusPembayaran` menjadi true.
- Nilai `statusPembayaran` hanya boleh dibaca (Read-Only) menggunakan getter, tidak boleh memiliki fungsi setter langsung dari luar kelas demi alasan keamanan transaksi.
- Uji kode Anda menggunakan kelas **TestBioskop** berikut:

```
1 public class TestBioskop {
2     public static void main(String[] args) {
3         Tiket tiket1 = new Tiket("Avengers: Endgame", -50000);
4         System.out.println("Film: " + tiket1.getJudulFilm());
5         System.out.println("Harga Tiket: " + tiket1.getHargaDasar());
6         System.out.println("Status Lunas? " + tiket1.isStatusPembayaran());
7
8         System.out.println("\nMemproses pembayaran...");
9         tiket1.lakukanPembayaran();
10        System.out.println("Status Lunas Terbaru? " + tiket1.isStatusPembayaran());
11    }
12 }
```

Jawaban: